Solving the Poisson Equation

Contents

- 1.1. Weak formulation
- 1.2. The Sobolev space H^1, linear and bilinear forms
- 1.3. The Finite Element Method
- 1.4. Poisson equation in NGSolve:
- 1.5. Visualizing the basis functions

The finite element method is a numerical method for solving partial differential equations approximately. A typical example is the Poisson equation:

$$-\Delta u(x) = f(x) \quad \forall\, x \in \Omega$$

The right hand side f is a given function, and we search for the solution u. The domain Ω is a subset of $\mathbb{R}^d$. The Poisson equation is a model for many physical phenomena:

- f can be a heat source distribution, and u is the temperature
- f can be an electric charge distribution, and u is the electrostatic potential

To select a unique solution u we have to specify boundary conditions, for example homogeneous Dirichlet boundary conditions

$$u(x) = 0 \quad \forall\, x \in \partial\Omega$$

1.1. Weak formulation

We derive the weak formulation (also called variational formulation) of the Poisson

equation. The formulation above is called the strong form. The weak form is the starting point for the finite element discretization method.

First, we multiply the Poisson equation by a so called test function. It is an arbitrary function, some restriction will be given later as needed. We multiply the strong form by the function v:

$$-\Delta u(x)v(x) = f(x)v(x) \qquad \forall x \in \Omega$$

We integrate over the domain Ω:

$$-\int_\Omega \Delta u(x)v(x)dx = \int_\Omega f(x)v(x)dx$$

From Gauss' Theorem $\int_\Omega \operatorname{div} b\, dx = \int_{\partial\Omega} n \cdot b\, ds$ applied to the vector field $b = \nabla u v$ we obtain

$$\int_{\partial\Omega} n\nabla u\, v\, ds = \int_\Omega \operatorname{div}(\nabla u\, v)\, dx = \int_\Omega \Delta u v + \nabla u \nabla v\, dx.$$

This allows us to rewrite the left hand side such that

$$\int_\Omega \nabla u \nabla v\, dx - \int_{\partial\Omega} \frac{\partial u}{\partial n} v\, ds = \int_\Omega f v\, dx$$

In the case of Dirichlet boundary conditions we allow only test-functions v such that $v(x) = 0$ on the boundary $\partial\Omega$.

We have derived the weak form: find u such that $u = 0$ on $\partial\Omega$ and

$$\int_\Omega \nabla u \nabla v\, dx = \int_\Omega f v\, dx$$

holds true for all test-functions v with $v = 0$ on $\partial\Omega$. Note that the weak formulation needs only first order derivatives of u and v, in contrast to the strong form which requires second order derivatives of u.

1.2. The Sobolev space H^1, linear and bilinear forms

The proper space to search for the solution is the so called Sobolev space

$$H^1(\Omega) := \{u \in L_2(\Omega) : \nabla u \in L_2(\Omega)^d\}$$

The super-script 1 indicates that we want to have first order derivatives in L_2. We just note that the derivative is understood in weak sense, which is well defined for functions with kinks. The vector space H^1 comes with the norm

$$\|u\|_{H^1}^2 := \|u\|_{L_2}^2 + \|\nabla u\|_{L_2}^2$$

and the inner product

$$(u, v)_{H^1} = (u, v)_{L_2} + (\nabla u, \nabla v)_{L_2}.$$

It is a complete space with an inner product what is called a Hilbert space.

It does not make sense to take boundary values of L_2-functions. The so called trace theorem tells us that boundary values of H^1 functions are well defined:

$$u_{|\partial\Omega} \in L_2(\partial\Omega)$$

Thus it makes sense to define the sub-space satisfying homogeneous Dirichlet boundary conditions

$$H_0^1(\Omega) = \{u \in H^1(\Omega) : u_{|\partial\Omega} = 0\}$$

Let us consider the term on the left hand side of the variational formulation:

$$A(u, v) := \int_\Omega \nabla u \nabla v \, dx$$

For given functions u and v from the Sobolev space, we compute the number $\int \nabla u \nabla v \, dx$. Thus, $A(.,.)$ is a function mapping from two elements from H^1 into $\mathbb{R}$:

$$A(.,.) : H^1 \times H^1 \to \mathbb{R}$$

The function $A(.,.)$ is linear in both arguments, and thus we call it a bilinear-form.

Similarly, the right hand side

$$f(v) := \int_\Omega f v \, dx$$

is a linear function

$$f(.) : H^1 \to \mathbb{R},$$

which we call a linear form. We use the same symbol for the function $f : \Omega \to \mathbb{R}$ and the linear-form $f : H^1(\Omega) \to \mathbb{R}$.

Having these objects defined, the weak formulation reads now

$$\text{find}\, u \in H_0^1 : \quad A(u,v) = f(v) \quad \forall\, v \in H_0^1$$

This abstract formalism in Hilbert spaces, bilinear and linear forms apply for a large class of (elliptic) partial differential equations.

1.3. The Finite Element Method

The weak formulation is the starting point for the finite element method. We cannot compute the solution in an infinite dimensional Hilbert space. But, we can define a finite dimensional sub-space

$$V_h \subset H_0^1$$

and restrict the weak formulation to V_h:

$$\text{find}\, u_h \in V_h : \quad A(u_h, v_h) = f(v_h) \quad \forall\, v_h \in V_h$$

This is called the Galerkin method. The finite element solution u_h is some approximation to the true solution u. We will analyze the discretization error $\|u - u_h\|_{H^1}$.

For computing the discrete solution u_h we have to choose a basis for the function space V_h, where $N = \dim V_h$

$$V_h = \text{span}\{p_1(x), \dots, p_N(x)\}$$

By means of this basis we can expand the solution u_h as

$$u_h(x) = \sum_{i=1}^{N} u_i p_i(x)$$

The coefficients u_i are combined to the coefficient vector $u = (u_1, \dots, u_N) \in \mathbb{R}^N$.

Instead of testing with all test-functions from V_h, by linearity of $A(.,.)$ and $f(.)$, it is enough to test only with the basis functions $p_j(x), j = 1, \dots, N$

Thus, the finite element problem can be rewritten as

$$\text{find}\, u \in \mathbb{R}^N : \quad A(\sum_i u_i p_i, p_j) = f(p_j) \qquad \forall\, j = 1, \dots N$$

By linearity of $A(.,.)$ in the first argument we can write

$$\text{find}\, u \in \mathbb{R}^N : \quad \sum_{i=1}^{N} A(p_i, p_j)\, u_i = f(p_j) \qquad \forall\, j = 1, \dots N$$

Since the basis functions are known, we can compute the matrix $A \in \mathbb{R}^{N \times N}$ with entries

$$A_{j,i} = A(p_i, p_j) = \int_\Omega \nabla p_i(x) \nabla p_j(x)\, dx$$

and the vector $f \in \mathbb{R}^N$ as

$$f_j = f(p_j) = \int_\Omega f(x) p_j(x)\, dx$$

Solving the finite element problem results in the linear system of equations for the coefficient vector $u = (u_1, \dots, u_N)$:

$$\text{find } u \in \mathbb{R}^N : \quad Au = f$$

By means of the coefficient vector, we have a representation of the finite element solution

$$u_h(x) = \sum_{i=1}^N u_i p_i(x)$$

1.4. Poisson equation in NGSolve:

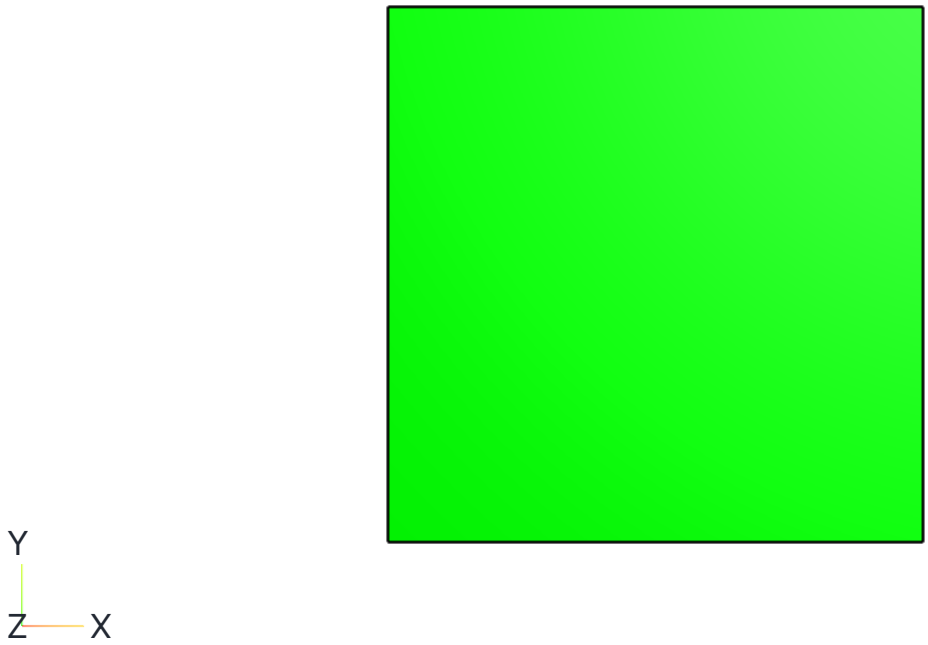
The Python interface to NGSolve allows us to enter the equation very close to its mathematical formulation.

```
# load Netgen/NGSolve
from ngsolve import *
from ngsolve.webgui import Draw
```

The unit-square $\Omega = (0,1)^2$ is a predefined domain in NGSolve:

```
Draw(unit_square.shape);
```

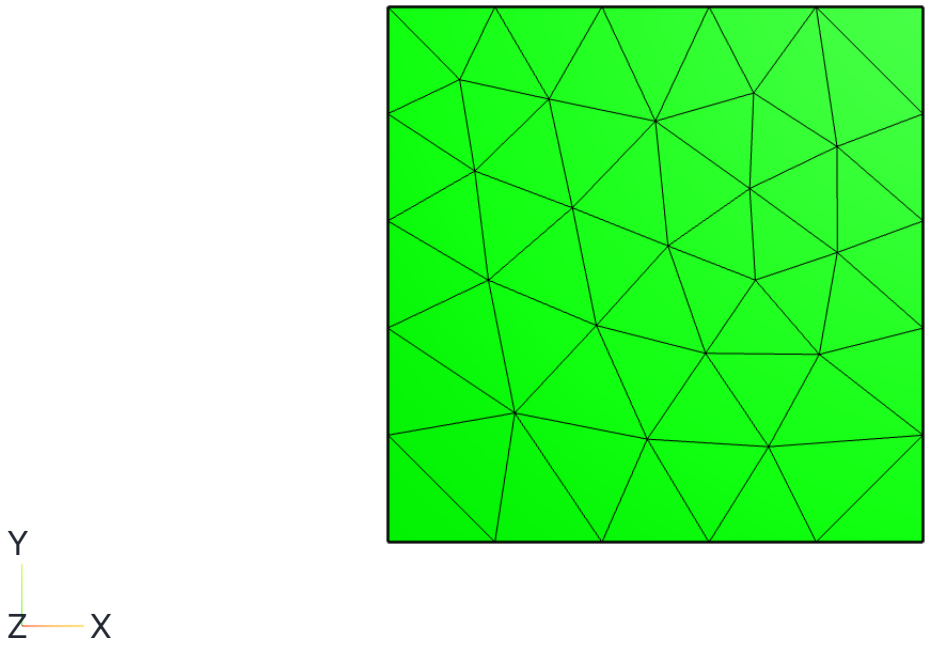
Open Controls



We generate a mesh (also called a triangulation) for Ω of mesh-size $h = 0.2$:

```
mesh = Mesh(unit_square.GenerateMesh(maxh=0.2))
Draw (mesh);
```

Open Controls



Number of vertices and elements:

```
mesh.nv, mesh.ne
```

```
(38, 54)
```

define the finite element space (dof is short for degree of freedom):

```
fes = H1(mesh, order=3, dirichlet=".*")
print ("number of dofs =", fes.ndof)
```

```
number of dofs = 274
```

Define the bilinear-form. We define it by means of trial- and test-functions of the space:

```
u = fes.TrialFunction()
v = fes.TestFunction()
a = BilinearForm(grad(u)*grad(v)*dx)
```

Similarly, we define the linear-form. The function `funcf` is defined by means of `x` and `y`, which are pre-defined symbols for Cartesian coordinates:

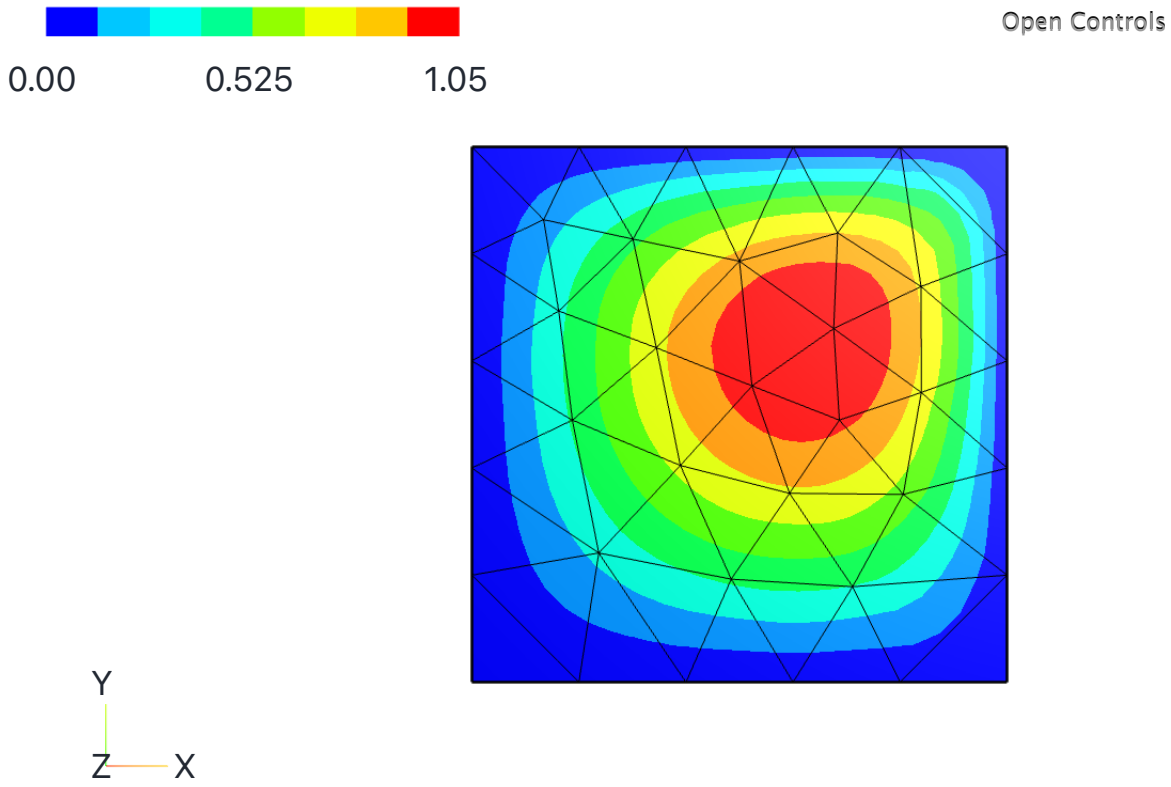
```
funcf = 50*x*y
f = LinearForm(funcf*v*dx)
```

compute the matrix and right hand side vector:

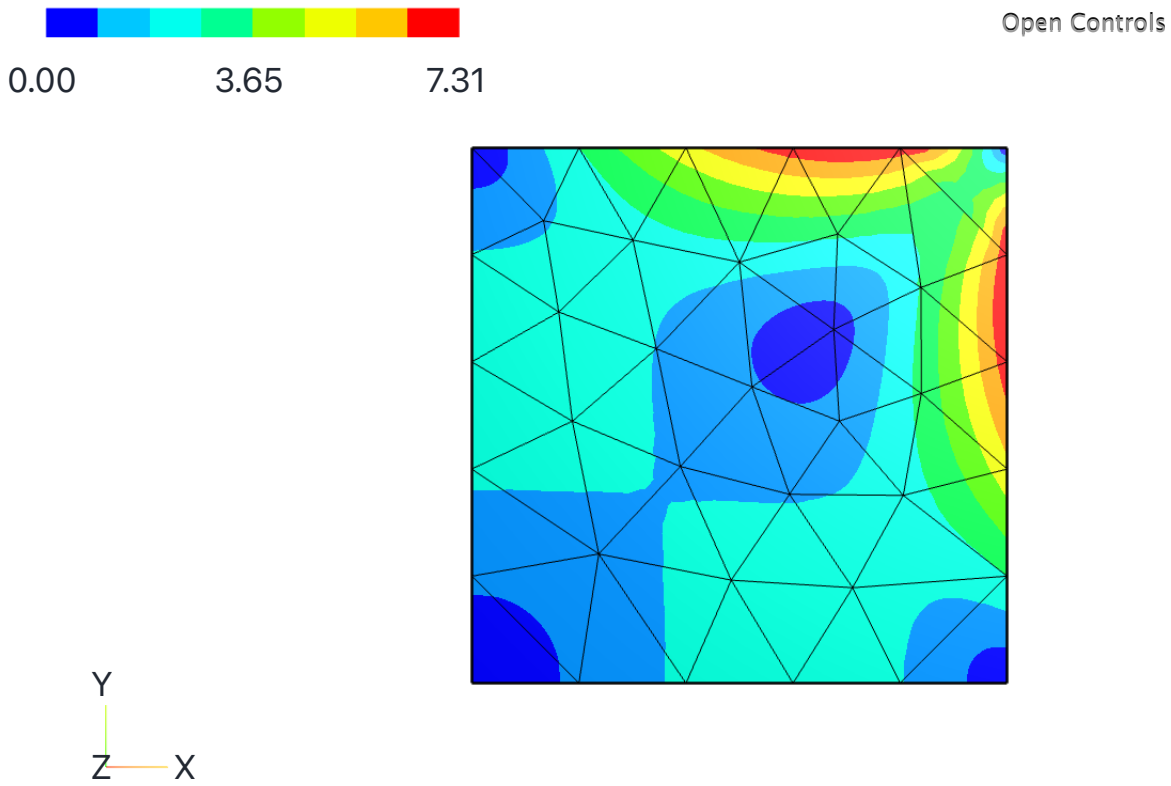
```
a.Assemble()
f.Assemble();
```

solve the linear system. Restrict the set of basis functions to non-Dirichlet basis functions (freedofs):

```
gfu = GridFunction(fes)
gfu.vec.data = a.mat.Inverse(freedofs=fes.FreeDofs()) * f.vec
Draw (gfu);
```




```
Draw (grad(gfu), mesh, order=3, vectors = True);
```



We can inspect the matrix entries. Either directly printing the matrix, or convert the sparse matrix to scipy:

```
# (i,j,val) = a.mat.COO()
# print (list(i),list(j),list(val))
# print (a.mat)
from scipy.sparse import csr_matrix
print (csr_matrix(a.mat.CSR()))
```

```
<Compressed Sparse Row sparse matrix of dtype 'float64'
        with 4282 stored elements and shape (274, 274)>
  Coords        Values
  (0, 0)        0.9999999999999964
  (0, 4)        -0.499999999999999
  (0, 19)       -0.4999999999999974
  (0, 38)       -0.08333333333333291
  (0, 39)       -2.662800535624399e-16
  (0, 40)       -0.08333333333333307
  (0, 41)       -3.1181654480683108e-16
  (0, 58)       0.16666666666666605
  (0, 59)       -4.85722573273506e-17
  (0, 220)      -5.204170427930421e-17
  (1, 1)        0.9999999999999961
  (1, 7)        -0.4999999999999979
  (1, 8)        -0.4999999999999982
```

```
(1, 42)       -0.0833333333333329
(1, 43)       -3.0531133177191805e-16
(1, 44)       -0.08333333333333301
(1, 45)       -2.6281060661048627e-16
(1, 74)       0.166666666666666
(1, 75)       5.551115123125783e-17
(1, 226)      -5.204170427930421e-17
(2, 2)        0.999999999999996
(2, 11)       -0.4999999999999978
(2, 12)       -0.4999999999999982
(2, 46)       -0.0833333333333329
(2, 47)       -3.1138286393783687e-16
:     :
(271, 206)    -0.0019075882630131035
(271, 207)    -0.00032462643307425555
(271, 218)    -0.006168208078166843
(271, 219)    -0.0017448330381255033
(271, 271)    0.010145255812379695
(272, 32)     -2.949029909160572e-17
(272, 33)     6.418476861114186e-17
(272, 37)     -3.469446951953614e-17
(272, 202)    -0.004571735817577506
(272, 203)    0.00038442360203229766
(272, 206)    -0.0043741138131356645
(272, 207)    0.00031854960055168445
(272, 210)    -0.0055273846192325626
(272, 211)    -6.587400148061159e-05
(272, 272)    0.009648822833297144
(273, 34)     -3.642919299551295e-17
(273, 36)     6.591949208711867e-17
(273, 37)     -2.42861286636753e-17
(273, 212)    -0.005539905484712292
(273, 213)    0.0004168482600137511
(273, 214)    -0.00386108577558625
(273, 215)    -0.000142758309694928
(273, 218)    -0.005111630555627509
(273, 219)    -0.0005596065697086791
(273, 273)    0.009675081210617355
```

the right hand side vector:

```
import numpy as np
print (np.array(f.vec))
```

```
[ 6.66666667e-04  1.60000000e-02  3.00666667e-01  2.50316962e-02
  1.53787128e-02  3.19680983e-02  3.64215571e-02  6.01137796e-02
  2.58401058e-01  3.41500143e-01  4.64175433e-01  7.54078336e-01
  9.65054998e-01  5.02685745e-01  3.65223563e-01  1.45178183e-01
  2.63488409e-02  3.03868260e-02  3.09439654e-02  1.51827952e-02
```

```
 1.57438188e-01  1.96522377e-01  2.73642628e-01  5.02716675e-01
 7.00784377e-01  1.07203103e+00  7.84223105e-01  8.62882720e-01
 4.14109207e-01  1.25387779e-01  1.78478512e-01  1.86485696e-01
 3.60521663e-01  3.18641431e-01  6.87560986e-01  4.04987970e-01
 3.88160967e-01  5.00017631e-01 -1.11111111e-04 -1.58730159e-05
-1.11111111e-04 -1.58730159e-05 -1.55555556e-03  1.58730159e-05
-3.22222222e-03 -5.39682540e-04 -3.67777778e-02  5.39682540e-04
-3.67777778e-02  5.39682540e-04 -2.94583153e-03 -3.71057295e-04
-6.65714424e-04  7.14041600e-06 -3.85020635e-03 -4.30399229e-04
-6.77560317e-04 -2.31247349e-05 -1.12348111e-03  4.69889843e-07
-2.59956630e-03 -4.04715869e-04 -7.64203095e-04 -1.47073262e-05
-3.89396024e-03 -4.08044554e-04 -4.16664199e-03 -5.79363310e-04
-9.33308210e-04 -1.26303694e-05 -4.39644983e-03 -5.24984536e-04
-4.82322749e-03 -6.82271737e-04 -8.71483484e-03 -1.28593920e-03
-6.91290333e-03 -7.96112164e-04 -1.20426957e-02 -5.24126690e-04
-1.64573389e-02  5.05626399e-04 -2.21569530e-02 -4.88847773e-04
-1.64293668e-02 -4.35267940e-04 -2.60499977e-02  6.02151177e-04
-3.02097864e-02 -2.52107903e-04 -2.29422764e-02 -4.36391360e-04
-3.48351637e-02  6.45625815e-04 -4.10186687e-02 -8.93599027e-05
-7.07762258e-02 -8.82145664e-06 -5.67938843e-02  9.61632700e-04
-2.26174899e-02  4.36933428e-04 -5.99814890e-02  1.02241565e-03
-4.81456663e-02  1.07899843e-03 -2.13152505e-02  5.75677602e-04
-4.04877136e-02  1.24423796e-04 -3.91936159e-02  1.10087461e-03
-1.04121804e-02  4.67186604e-04 -3.43058966e-02  3.85899424e-05
-2.42403790e-02  7.62880882e-04 -1.47307067e-02 -2.01059452e-04
-8.20464635e-03  3.03812690e-04 -7.66757639e-04  1.04659738e-05
-3.07655862e-03 -4.16268218e-04 -3.37430318e-03 -4.45285933e-04
-7.30230341e-04  1.40081000e-05 -3.82716652e-03 -5.31338127e-04
-3.77311819e-03 -4.67341551e-04 -6.57227569e-04  2.23174630e-05
-3.79051835e-03 -4.01377404e-04 -4.02598429e-03 -5.60508558e-04
-2.56353334e-03 -4.00674040e-04 -8.05766635e-03 -2.42271640e-04
-7.74031116e-03 -2.24968428e-04 -1.00791015e-02 -6.59789939e-04
-9.02992762e-03 -1.83113660e-04 -1.18282465e-02 -6.17006547e-04
-1.14383060e-02 -3.78599836e-04 -1.79672804e-02 -9.24686080e-04
-1.35048754e-02 -4.26102844e-04 -2.40941811e-02 -7.51249781e-04
-1.65946517e-02  3.31795257e-04 -1.85553209e-02 -2.05836605e-04
-3.57098243e-02 -7.37663402e-04 -2.83143537e-02  3.36784018e-05
-2.19095184e-02  4.42354897e-04 -4.13523565e-02  2.19592138e-04
-3.34913520e-02  7.32812750e-04 -3.38935650e-02  8.34464680e-04
-3.22556427e-02  5.34677713e-04 -2.55831924e-02  7.71255906e-04
-3.09659540e-02 -2.55806818e-04 -2.41303394e-02  9.65516668e-04
-2.75495542e-02  5.59631071e-04 -9.71319573e-03  4.86341176e-04
-1.13638364e-02  5.81790841e-04 -1.80394336e-02  1.72369402e-04
-6.41052633e-03  1.22793076e-05 -7.67492196e-03  1.00764423e-04
-1.19932499e-02 -4.95707252e-04 -1.14829084e-02 -3.89942772e-04
-1.17837223e-02 -6.50554480e-04 -1.46537623e-02  2.69401360e-04
-1.64821908e-02 -5.10036723e-04 -1.69210182e-02 -3.60347220e-04
-1.49298923e-02 -3.02891766e-04 -1.68205357e-02 -6.73833256e-04
-2.17665222e-02  4.13062695e-04 -2.49876307e-02  7.14952120e-04
-2.04885052e-02 -3.93253619e-04 -1.82794918e-02  1.76532291e-04
 6.34920635e-05  4.42017576e-04  5.07957962e-04  1.15461890e-03
 6.23153294e-04  1.29393565e-03  1.04761905e-03  2.36134197e-03
 5.98627148e-03  4.78896991e-03  3.32170989e-03  6.49537749e-03
 5.57905819e-03  9.03906735e-03  7.40289876e-03  1.45079365e-02
```

```
  8.81839455e-03  8.27205676e-03  7.35939638e-03  4.06746173e-03
  5.57663961e-03  1.20536735e-03  1.98500156e-03  4.42084876e-04
  5.10697200e-04  7.96464045e-04  4.85830721e-04  1.02287770e-03
  4.29433138e-04  4.89856095e-04  1.37504318e-02  1.01859919e-02
  1.12369013e-03  2.26844957e-03  2.06353279e-03  2.47084464e-03
  6.83618837e-03  6.50679687e-03  4.05481214e-03  3.17281726e-03
  1.81116416e-03  2.73100280e-03  4.58286836e-03  6.36719397e-03
  2.06100470e-03  1.96229633e-03  2.62677007e-03  4.08567879e-03
  3.37042767e-03  4.96655092e-03  4.59979983e-03  3.29096347e-03
  3.40400909e-03  4.04680263e-03]
```

and the solution vector:

```
print (np.array(gfu.vec))
```

```
[ 0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  2.95604147e-01  4.15960317e-01  3.90617436e-01  6.05046764e-01
  7.39473079e-01  7.67810331e-01  7.89973079e-01  8.32999355e-01
  4.92246882e-01  1.93740634e-01  3.63451645e-01  4.08449548e-01
  7.65881651e-01  6.89386988e-01  1.04654230e+00  7.52519575e-01
  9.49056407e-01  9.82901183e-01  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  0.00000000e+00  0.00000000e+00
  0.00000000e+00  0.00000000e+00  3.29872146e-01 -5.63346048e-02
  0.00000000e+00  0.00000000e+00 -4.42881775e-01  9.31354352e-06
  4.75238124e-02 -1.66538938e-02  0.00000000e+00  0.00000000e+00
 -3.98310110e-01 -3.27434699e-02  5.23897743e-02 -2.10571595e-02
  0.00000000e+00  0.00000000e+00 -1.10719879e-01 -3.29834403e-02
 -9.47839680e-02 -3.42461548e-02 -9.15861624e-01 -1.69462094e-01
  8.20917155e-02 -3.03728259e-02  0.00000000e+00  0.00000000e+00
 -6.98413007e-01  9.16653462e-02  3.54891894e-02 -1.09369126e-01
  0.00000000e+00  0.00000000e+00 -7.27712190e-01  7.29463845e-02
 -2.64963388e-01 -7.63522783e-02  0.00000000e+00  0.00000000e+00
 -6.85851125e-01  5.90090450e-02 -7.77840564e-01 -7.42668756e-02
 -2.92207757e+00  5.24643091e-04 -6.40175452e-01  6.06300895e-02
  0.00000000e+00  0.00000000e+00 -1.73694228e+00  3.65154454e-01
 -6.37547720e-01  2.58163508e-02  0.00000000e+00  0.00000000e+00
 -6.64577861e-01 -4.72275531e-03 -1.15806042e+00  1.34960788e-01
  0.00000000e+00  0.00000000e+00 -5.95762162e-01 -3.62105143e-02
 -7.26042508e-01  7.56773654e-02 -1.28516068e-02 -5.34212630e-02
 -3.24095771e-01  3.12510342e-02  0.00000000e+00  0.00000000e+00
 -1.26877324e-01 -1.12759632e-02  8.33634076e-02 -3.39581436e-02
  0.00000000e+00  0.00000000e+00 -6.18908383e-02 -2.24336757e-02
 -1.00848552e-01 -2.99981650e-02  0.00000000e+00  0.00000000e+00
```

```
-3.79668370e-01 -3.13657047e-02  5.55076801e-02 -2.12177498e-02
 5.70778139e-02 -1.63965441e-02 -2.21544110e-01 -6.22121768e-03
-2.21899082e-01 -5.57503828e-03  4.19449329e-02 -3.62805393e-02
-2.01711220e-01 -1.27187408e-02 -7.66285082e-02 -2.52958866e-02
-2.63371926e-01 -2.02115464e-02 -2.37324988e-01 -2.47009316e-02
-1.30405667e-01 -3.67623540e-02 -1.79028328e-01 -9.09214506e-03
-4.09258552e-01  3.64980822e-02 -1.81733686e-01 -3.11306479e-02
-2.71110392e-01 -3.89729544e-02 -5.28253586e-01 -6.85582235e-03
-3.66380067e-01  3.22674611e-02 -6.33012197e-01  4.93768884e-02
-4.10254930e-01  2.59790767e-02 -2.61525964e-01  9.24959888e-03
-4.74136171e-01  4.98542513e-02 -8.32618495e-02  3.00884365e-02
-4.75412215e-01 -4.74015964e-02 -4.27020463e-01  5.29345768e-02
-5.96710879e-01  6.18100068e-02  3.71540991e-02  1.14675072e-02
-2.94215188e-01  4.08902292e-02 -3.37191234e-01  8.10445553e-03
-9.81699465e-02 -4.63529688e-03 -1.54585327e-01  3.12984070e-03
-8.27832592e-02 -2.31560473e-02 -2.22664755e-01 -1.80640914e-02
-1.06301837e-01 -3.51362555e-02 -2.67356721e-01  1.34974081e-02
-1.59765535e-01 -1.57306664e-02 -2.85415895e-01 -2.82880114e-02
-3.06095382e-01 -1.83505198e-02 -1.52044789e-01 -3.14875856e-02
-2.35718773e-01  2.54378129e-02 -2.81698084e-01  2.24256670e-02
-2.20495477e-01 -2.30627626e-02 -2.82021838e-01  1.32939301e-02
 5.71428571e-03  1.11595660e-02  3.48536424e-02 -2.97322554e-02
 6.71449475e-02 -6.43274098e-02  9.42857143e-02 -2.35704148e-01
-2.32458798e-03  2.55544240e-02  2.49965015e-02  7.60477223e-02
-1.66356424e-02  2.08051658e-01 -1.33567475e-01  1.30571429e+00
 2.10361863e-01  1.56308884e-01 -9.98425186e-02  4.82379867e-02
-1.19413559e-01 -3.72920325e-03 -4.05321436e-02  7.67601758e-02
 6.10119912e-02 -5.00546489e-02  3.50628659e-02 -4.75350122e-02
 1.16588198e-02 -9.52633465e-03 -5.75024994e-01 -9.10938369e-02
-3.11297701e-02  5.18073298e-02  2.09715211e-02 -1.83841932e-02
 1.07953045e-01 -7.82941876e-02  1.25484570e-01 -5.47856986e-02
 5.11490841e-02 -6.55129325e-02  7.97346584e-02  5.71733035e-02
 4.60455061e-02  2.36267936e-02 -3.06062272e-02  1.90977442e-02
 3.22162780e-02 -6.04780687e-02 -5.84153211e-02 -9.63680712e-03
 9.80651757e-03  2.18838590e-02]
```

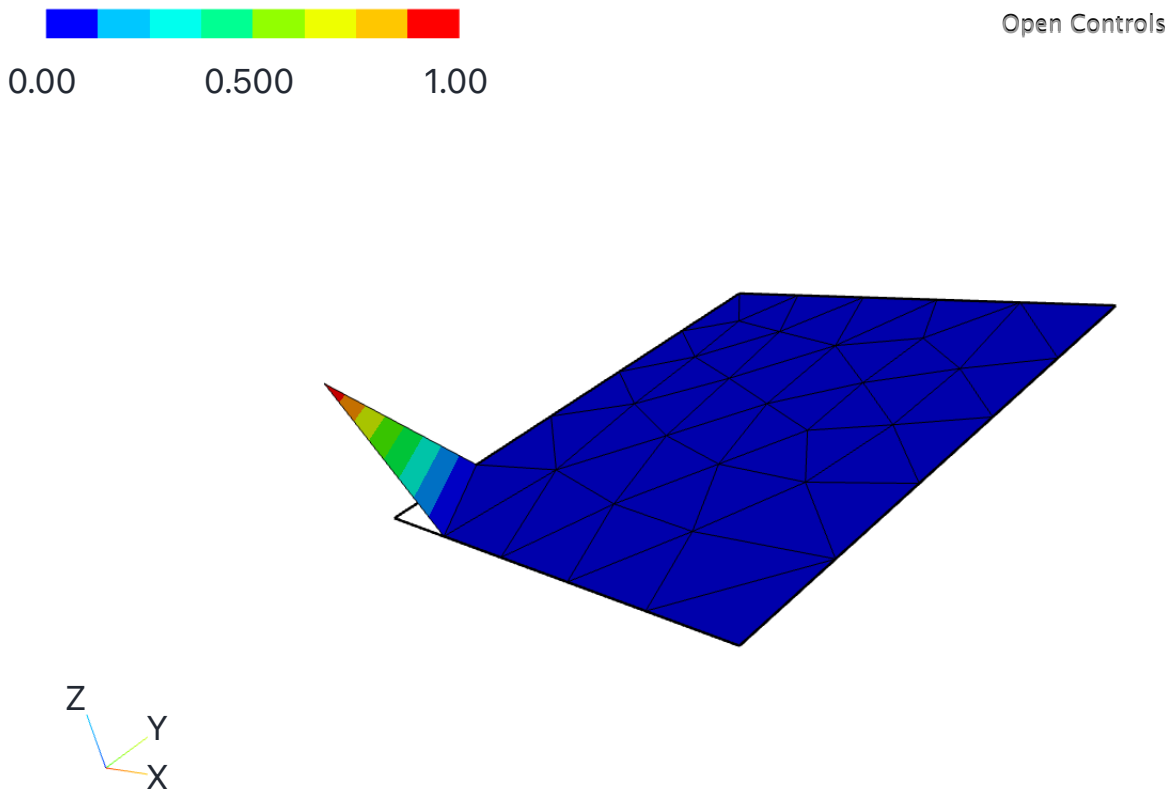
1.5. Visualizing the basis functions

For a finite element space of order=1, the basis-functions are associated with mesh vertices. For higher order, we also have edge and face based basis-functions.

Let's have a look at the basis functions. Well know are the so-called hat-functions, which have value 1 in one vertex, 0 in all other, and a linear interpolation in between.

Open controls, and use the `multidim` slider to select basis function

```
gf = GridFunction(fes, multidim=mesh.nv)
for i in range (mesh.nv):
    gf.vecs[i][:] = 0
    gf.vecs[i][i] = 1
scene = Draw (gf, mesh, deformation=True, animate=True, scale=0.3, euler_
```

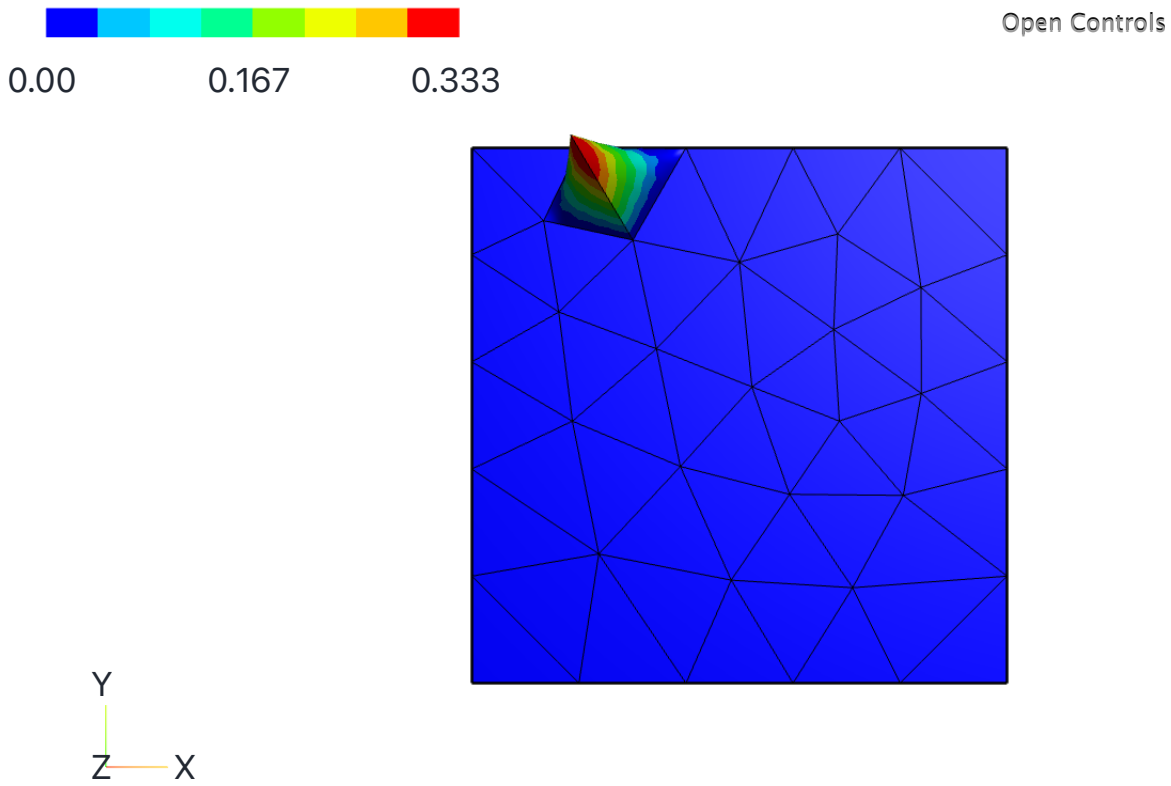


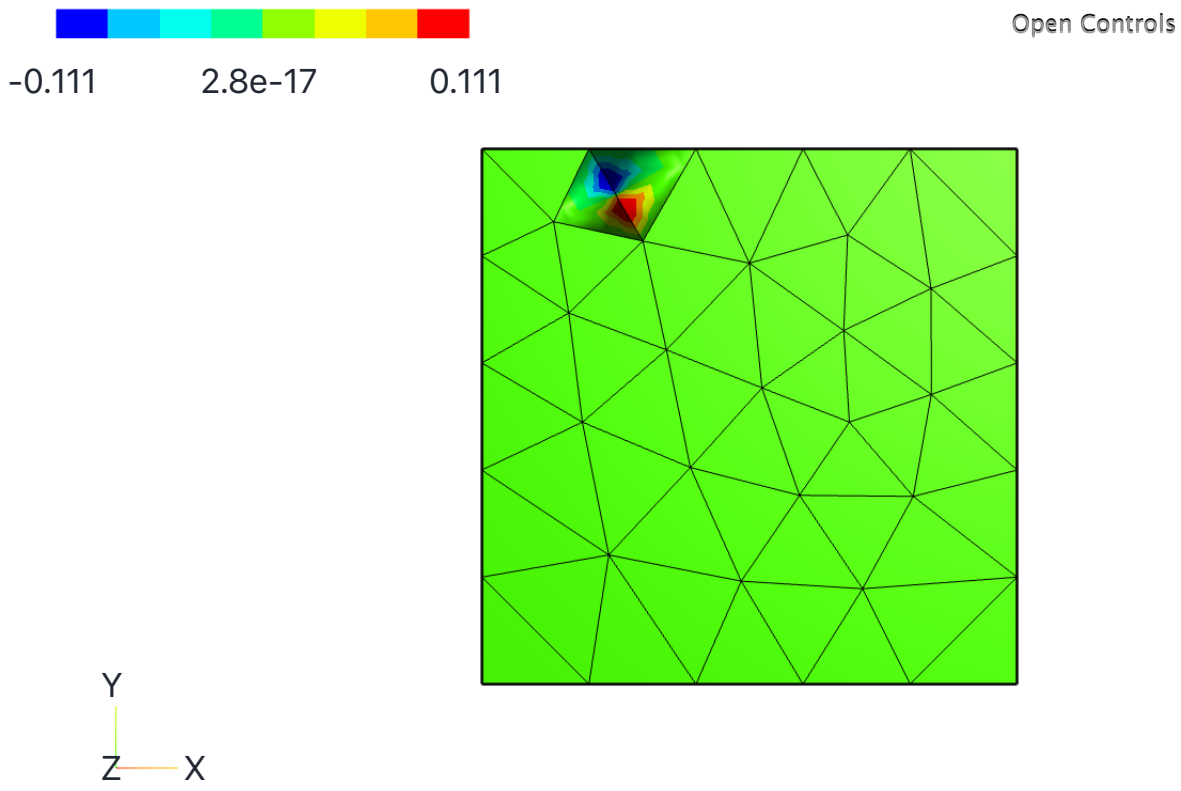
For higher order spaces there are also basis functions on edges:

```
edgedofs = fes.GetDofNrs(NodeId(EDGE,40))
print ("dofs on edge #40: ", edgedofs)

gf = GridFunction(fes)
for edof in edgedofs:
    gf.vec[:] = 0
    gf.vec[edof] = 1
    Draw (-3*gf, mesh, deformation=True, order=3)
```

```
dofs on edge #40:  (118, 119)
```



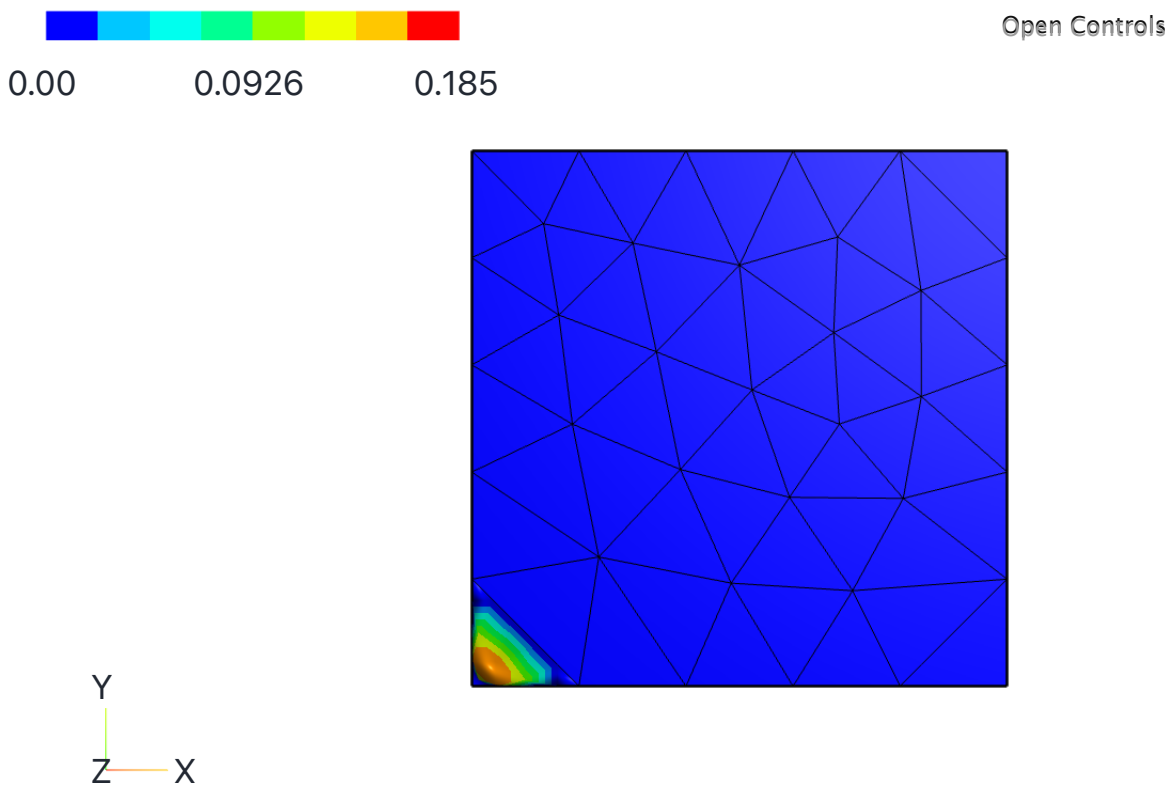


and also basis functions associated with faces:

```
facedofs = fes.GetDofNrs(NodeId(FACE,0))
print ("dofs on face #0: ", facedofs)

for fdof in facedofs:
    gf.vec[:] = 0
    gf.vec[fdof] = 1
    Draw (5*gf, mesh, deformation=True, order=3)
```

```
dofs on face #0:  (220,)
```



To get help on types or objects use the help function:

```
help (Mesh)
```

```
Help on class Mesh in module ngsolve.comp:

class Mesh(pybind11_builtins.pybind11_object)
 |  NGSolve interface to the Netgen mesh. Provides access and functionali
 |  to use the mesh for finite element calculations.
 |
 |  Parameters:
 |
 |  mesh (netgen.Mesh): a mesh generated from Netgen
```

```
 Method resolution order:
     Mesh
     pybind11_builtins.pybind11_object
     builtins.object

 Methods defined here:

 BBBoundaries(...)
     BBBoundaries(self: ngsolve.comp.Mesh, pattern: str) -> ngsolve.cor

     Return co dim 3 boundary mesh-region matching the given regex pat

 BBoundaries(...)
     BBoundaries(self: ngsolve.comp.Mesh, pattern: str) -> ngsolve.comp

     Return co dim 2 boundary mesh-region matching the given regex pat

 Boundaries(...)
     Boundaries(*args, **kwargs)
     Overloaded function.

     1. Boundaries(self: ngsolve.comp.Mesh, pattern: str) -> ngsolve.co

     Return boundary mesh-region matching the given regex pattern

     2. Boundaries(self: ngsolve.comp.Mesh, bnds: collections.abc.Seque

     Generate boundary mesh-region by boundary condition numbers

 BoundaryCF(...)
     BoundaryCF(self: ngsolve.comp.Mesh, values: dict, default: ngsolve

     Boundary wise CoefficientFunction.
     First argument is a dict from either boundary names or Region obje
     CoefficientFunction (-values). Later given names/regions override
     values. Optional last argument (default) is the value for not give
     >>> penalty = mesh.BoundaryCF({ "top" : 1e6 }, default=0)
     will create a CF being 1e6 on the top boundary and 0. elsewhere.

 BuildRefinementTree(...)
     BuildRefinementTree(self: ngsolve.comp.Mesh) -> pyngcore.pyngcore.

 Contains(...)
     Contains(self: ngsolve.comp.Mesh, x: typing.SupportsFloat | typing

     Check if the point (x,y,z) is in the meshed domain (is inside a vo

 Curve(...)
     Curve(self: ngsolve.comp.Mesh, order: typing.SupportsInt | typing.

     Curve the mesh elements for geometry approximation of given order

 Elements(...)
```

```
|      Elements(self: ngsolve.comp.Mesh, VOL_or_BND: ngsolve.comp.VorB =
|
|      Return an iterator over elements on VOL/BND
|
|  GeoParamCF(...)
|      GeoParamCF(self: ngsolve.comp.Mesh) -> ngsolve.fem.CoefficientFunc
|
|  GetBBBoundaries(...)
|      GetBBBoundaries(self: ngsolve.comp.Mesh) -> tuple
|
|      Return list of boundary conditions for co dimension 3
|
|  GetBBoundaries(...)
|      GetBBoundaries(self: ngsolve.comp.Mesh) -> tuple
|
|      Return list of boundary conditions for co dimension 2
|
|  GetBoundaries(...)
|      GetBoundaries(self: ngsolve.comp.Mesh) -> tuple
|
|      Return list of boundary condition names
|
|  GetCurveOrder(...)
|      GetCurveOrder(self: ngsolve.comp.Mesh) -> int
|
|  GetHPElementLevel(...)
|      GetHPElementLevel(self: ngsolve.comp.Mesh, ei: ngsolve.comp.Elemer
|
|      THIS FUNCTION IS WIP!
|       Return HP-refinement level of element
|
|  GetMaterials(...)
|      GetMaterials(self: ngsolve.comp.Mesh) -> tuple
|
|      Return list of material names
|
|  GetNE(...)
|      GetNE(self: ngsolve.comp.Mesh, arg0: ngsolve.comp.VorB) -> int
|
|      Return number of elements of codimension VorB.
|
|  GetPMLTrafo(...)
|      GetPMLTrafo(self: ngsolve.comp.Mesh, dom: typing.SupportsInt | typ
|
|      Return pml transformation on domain dom
|
|  GetPMLTrafos(...)
|      GetPMLTrafos(self: ngsolve.comp.Mesh) -> list
|
|      Return list of pml transformations
|
|  GetParentElement(...)
|      GetParentElement(self: ngsolve.comp.Mesh, ei: ngsolve.comp.Element
|
```

```
|      Return parent element id on refined mesh
|
|  GetParentFaces(...)
|      GetParentFaces(self: ngsolve.comp.Mesh, fnum: typing.SupportsInt
|
|      Return parent faces
|
|  GetParentVertices(...)
|      GetParentVertices(self: ngsolve.comp.Mesh, vnum: typing.SupportsIr
|
|      Return parent vertex numbers on refined mesh
|
|  GetPeriodicNodePairs(...)
|      GetPeriodicNodePairs(self: ngsolve.comp.Mesh, arg0: ngsolve.fem.N(
|
|      returns list of periodic nodes with their identification number as
|
|  GetTrafo(...)
|      GetTrafo(self: ngsolve.comp.Mesh, eid: ngsolve.comp.ElementId) ->
|
|      returns element transformation of given element id
|
|  LocalHCF(...)
|      LocalHCF(self: ngsolve.comp.Mesh) -> ngsolve.fem.CoefficientFunct:
|
|  MapToAllElements(...)
|      MapToAllElements(*args, **kwargs)
|      Overloaded function.
|
|      1. MapToAllElements(self: ngsolve.comp.Mesh, arg0: ngsolve.fem.Int
|
|      2. MapToAllElements(self: ngsolve.comp.Mesh, rules: collections.ab
|
|  MaterialCF(...)
|      MaterialCF(self: ngsolve.comp.Mesh, values: dict, default: ngsolve
|
|      Domain wise CoefficientFunction.
|      First argument is a dict from either material names or Region obje
|      CoefficientFunction (-values). Later given names/regions override
|      values. Optional last argument (default) is the value for not give
|      >>> sigma = mesh.MaterialCF({ "steel_.*" : 2e6 }, default=0)
|      will create a CF being 2e6 on all domains starting with 'steel_' a
|
|  Materials(...)
|      Materials(*args, **kwargs)
|      Overloaded function.
|
|      1. Materials(self: ngsolve.comp.Mesh, pattern: str) -> ngsolve.con
|
|      Return mesh-region matching the given regex pattern
|
|      2. Materials(self: ngsolve.comp.Mesh, domains: collections.abc.Sec
|
|      Generate mesh-region by domain numbers
```

```
|
|  Refine(...)
|      Refine(self: ngsolve.comp.Mesh, mark_surface_elements: bool = Fals
|
|      Local mesh refinement based on marked elements, uses element-bisec
|
|  RefineFromTree(...)
|      RefineFromTree(self: ngsolve.comp.Mesh, arg0: pyngcore.pyngcore.Ar
|
|  RefineHP(...)
|      RefineHP(self: ngsolve.comp.Mesh, levels: typing.SupportsInt | typ
|
|      Geometric mesh refinement towards marked vertices and edges, uses
|
|  Region(...)
|      Region(self: ngsolve.comp.Mesh, vb: ngsolve.comp.VorB, pattern: st
|
|      Return boundary mesh-region matching the given regex pattern
|
|  RegionCF(...)
|      RegionCF(self: ngsolve.comp.Mesh, VorB: ngsolve.comp.VorB, value:
|
|      Region wise CoefficientFunction.
|      First argument is VorB, defining the co-dimension,
|      second argument is a dict from either region names or Region objec
|      CoefficientFunction (-values). Later given names/regions override
|      values. Optional last argument (default) is the value for not give
|      >>> sigma = mesh.RegionCF(VOL, { "steel_.*" : 2e6 }, default=0)
|      will create a CF being 2e6 on all domains starting with 'steel_' a
|
|  SetDeformation(...)
|      SetDeformation(self: ngsolve.comp.Mesh, gf: ngcomp::GridFunction)
|
|      Deform the mesh with the given GridFunction
|
|  SetElementOrder(...)
|      SetElementOrder(self: ngsolve.comp.Mesh, eid: ngsolve.comp.Element
|
|      For backward compatibility, not recommended to use
|
|  SetElementOrders(...)
|      SetElementOrders(self: ngsolve.comp.Mesh, eid: ngsolve.comp.Elemer
|
|      Set anisotropic element order (expert only)
|
|  SetPML(...)
|      SetPML(self: ngsolve.comp.Mesh, pmltrafo: ngsolve.comp.pml.PML, de
|
|      Set PML transformation on domain
|
|  SetRefinementFlag(...)
|      SetRefinementFlag(self: ngsolve.comp.Mesh, ei: ngsolve.comp.Elemer
|
|      Set refinementflag for mesh-refinement
```

```
|
|  SetRefinementFlags(...)
|      SetRefinementFlags(self: ngsolve.comp.Mesh, refine: collections.ab
|
|      Set refinementflags for mesh-refinement
|
|  SplitElements_Alfeld(...)
|      SplitElements_Alfeld(self: ngsolve.comp.Mesh) -> None
|
|  UnSetPML(...)
|      UnSetPML(self: ngsolve.comp.Mesh, definedon: object) -> None
|
|      Unset PML transformation on domain
|
|  UnsetDeformation(...)
|      UnsetDeformation(self: ngsolve.comp.Mesh) -> None
|
|      Unset the deformation
|
|  __call__(...)
|      __call__(self: ngsolve.comp.Mesh, x: typing.Annotated[numpy.typing
|
|      Get a MappedIntegrationPoint in the point (x,y,z) on the matching
|
|  __eq__(...)
|      __eq__(self: ngsolve.comp.Mesh, mesh: ngsolve.comp.Mesh) -> bool
|
|  __getitem__(...)
|      __getitem__(*args, **kwargs)
|      Overloaded function.
|
|      1. __getitem__(self: ngsolve.comp.Mesh, arg0: ngsolve.comp.Element
|
|      Return Ngs_Element from given ElementId
|
|      2. __getitem__(self: ngsolve.comp.Mesh, arg0: ngsolve.comp.NodeId)
|
|      Return MeshNode from given NodeId
|
|  __getstate__(...)
|      __getstate__(self: ngsolve.comp.Mesh, /) -> tuple[list]
|
|  __init__(...)
|      __init__(*args, **kwargs)
|      Overloaded function.
|
|      1. __init__(self: ngsolve.comp.Mesh, ngmesh: netgen.libngpy._mesh:
|
|      Make an NGSolve-mesh from a Netgen-mesh
|
|      2. __init__(self: ngsolve.comp.Mesh, filename: str, comm: pyngcore
|
|      Load a mesh from file.
|      In MPI-parallel mode the mesh is distributed over the MPI-group g:
```

```
 Ⅰ
 Ⅰ  __setstate__(...)
 Ⅰ      __setstate__(self: ngsolve.comp.Mesh, arg0: tuple) -> None
 Ⅰ
 Ⅰ  nnodes(...)
 Ⅰ      nnodes(self: ngsolve.comp.Mesh, arg0: ngsolve.fem.NODE_TYPE) -> in
 Ⅰ
 Ⅰ      number of nodes given type
 Ⅰ
 Ⅰ  nodes(...)
 Ⅰ      nodes(self: ngsolve.comp.Mesh, node_type: ngsolve.fem.NODE_TYPE) -
 Ⅰ
 Ⅰ      iterable of mesh nodes of type node_type
 Ⅰ
 Ⅰ  ----------------------------------------------------------------------
 Ⅰ  Readonly properties defined here:
 Ⅰ
 Ⅰ  comm
 Ⅰ      MPI-communicator the Mesh lives in
 Ⅰ
 Ⅰ  dim
 Ⅰ      mesh dimension
 Ⅰ
 Ⅰ  edges
 Ⅰ      iterable of mesh edges
 Ⅰ
 Ⅰ  faces
 Ⅰ      iterable of mesh faces
 Ⅰ
 Ⅰ  facets
 Ⅰ      iterable of mesh facets
 Ⅰ
 Ⅰ  levels
 Ⅰ      multigrid levels
 Ⅰ
 Ⅰ  ne
 Ⅰ      number of volume elements
 Ⅰ
 Ⅰ  nedge
 Ⅰ      number of edges
 Ⅰ
 Ⅰ  nface
 Ⅰ      number of faces
 Ⅰ
 Ⅰ  nfacet
 Ⅰ      number of facets
 Ⅰ
 Ⅰ  ngmesh
 Ⅰ      the Netgen mesh
 Ⅰ
 Ⅰ  nv
 Ⅰ      number of vertices
 Ⅰ
 Ⅰ  vertices
```

```
 |      iterable of mesh vertices
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors defined here:
 |
 |  __dict__
 |
 |  deformation
 |      mesh deformation
 |
 |  ----------------------------------------------------------------------
 |  Data and other attributes defined here:
 |
 |  __hash__ = None
 |
 |  ----------------------------------------------------------------------
 |  Static methods inherited from pybind11_builtins.pybind11_object:
 |
 |  __new__(*args, **kwargs) class method of pybind11_builtins.pybind11_ob
 |      Create and return a new object.  See help(type) for accurate signa
```

```
help (fes)
```

```
Help on H1 in module ngsolve.comp object:

class H1(FESpace)
 |  An H1-conforming finite element space.
 |
 |  The H1 finite element space consists of continuous and
 |  element-wise polynomial functions. It uses a hierarchical (=modal)
 |  basis built from integrated Legendre polynomials on tensor-product ele
 |  and Jaboci polynomials on simplicial elements.
 |
 |  Boundary values are well defined. The function can be used directly on
 |  boundary, using the trace operator is optional.
 |
 |  The H1 space supports variable order, which can be set individually fo
 |  faces and cells.
 |
 |  Internal degrees of freedom are declared as local dofs and are elimina
 |  if static condensation is on.
 |
 |  The wirebasket consists of all vertex dofs. Optionally, one can inclu
 |  first (the quadratic bubble) edge basis function, or all edge basis fu
 |  into the wirebasket.
 |
 |  Keyword arguments can be:
 |
 |  order: int = 1
 |    order of finite element space
```

```
|  complex: bool = False
|    Set if FESpace should be complex
|  dirichlet: regexpr
|    Regular expression string defining the dirichlet boundary.
|    More than one boundary can be combined by the | operator,
|    i.e.: dirichlet = 'top|right'
|  dirichlet_bbnd: regexpr
|    Regular expression string defining the dirichlet bboundary,
|    i.e. points in 2D and edges in 3D.
|    More than one boundary can be combined by the | operator,
|    i.e.: dirichlet_bbnd = 'top|right'
|  dirichlet_bbbnd: regexpr
|    Regular expression string defining the dirichlet bbboundary,
|    i.e. points in 3D.
|    More than one boundary can be combined by the | operator,
|    i.e.: dirichlet_bbbnd = 'top|right'
|  definedon: Region or regexpr
|    FESpace is only defined on specific Region, created with mesh.Mater:
|    or mesh.Boundaries('regexpr'). If given a regexpr, the region is ass
|    mesh.Materials('regexpr').
|  dim: int = 1
|    Create multi dimensional FESpace (i.e. [H1]^3)
|  dgjumps: bool = False
|    Enable discontinuous space for DG methods, this flag is needed for D
|    since the dofs have a different coupling then and this changes the s
|    pattern of matrices.
|  autoupdate: bool = False
|    Automatically update on a change to the mesh.
|  low_order_space: bool = True
|    Generate a lowest order space together with the high-order space,
|    needed for some preconditioners.
|  hoprolongation: bool = False
|    Create high order prolongation operators,
|    only available for H1 and L2 on simplicial meshes
|  order_policy: ORDER_POLICY = ORDER_POLICY.OLDSTYLE
|    CONSTANT .. use the same fixed order for all elements,
|    NODAL ..... use the same order for nodes of same shape,
|    VARIABLE ... use an individual order for each edge, face and cell,
|    OLDSTYLE .. as it used to be for the last decade
|  print: bool = False
|    (historic) print some output into file set by 'SetTestoutFile'
|  wb_withedges: bool = true(3D) / false(2D)
|    use lowest-order edge dofs for BDDC wirebasket
|  wb_fulledges: bool = false
|    use all edge dofs for BDDC wirebasket
|  hoprolongation: bool = false
|    (experimental, only trigs) creates high order prolongation,
|    and switches off low-order space
|
|  Method resolution order:
|      H1
|      FESpace
|      NGS_Object
|      pybind11_builtins.pybind11_object
```

```
 |      builtins.object
 |
 |  Methods defined here:
 |
 |  __getstate__(...)
 |      __getstate__(self: ngsolve.comp.FESpace, /) -> tuple
 |
 |  __init__(...)
 |      __init__(self: ngsolve.comp.H1, mesh: ngsolve.comp.Mesh, **kwargs)
 |
 |  __setstate__(...)
 |      __setstate__(self: ngsolve.comp.H1, arg0: tuple) -> None
 |
 |  ----------------------------------------------------------------------
 |  Static methods defined here:
 |
 |  __flags_doc__(...) method of pybind11_builtins.pybind11_detail_functio
 |      __flags_doc__() -> dict
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors defined here:
 |
 |  __dict__
 |
 |  ----------------------------------------------------------------------
 |  Methods inherited from FESpace:
 |
 |  ApplyM(...)
 |      ApplyM(self: ngsolve.comp.FESpace, vec: ngsolve.la.BaseVector, rho
 |
 |      Apply mass-matrix. Available only for L2-like spaces
 |
 |  ConvertL2Operator(...)
 |      ConvertL2Operator(self: ngsolve.comp.FESpace, l2space: ngsolve.com
 |
 |  CouplingType(...)
 |      CouplingType(self: ngsolve.comp.FESpace, dofnr: typing.SupportsInt
 |
 |
 |              Get coupling type of a degree of freedom.
 |
 |      Parameters:
 |
 |      dofnr : int
 |        input dof number
 |
 |  CreateDirectSolverCluster(...)
 |      CreateDirectSolverCluster(self: ngsolve.comp.FESpace, **kwargs) ->
 |
 |  CreateSmoothingBlocks(...)
 |      CreateSmoothingBlocks(self: ngsolve.comp.FESpace, **kwargs) -> pyn
 |
 |
 |      Create table of smoothing blocks for block-Jacobi/block-Gauss-Seid
```

```
|
|      Every table entry describes the set of dofs belonging a Jacobi/Gau
|
|      Paramters:
|
|      blocktype: string | [ string ] | int
|          describes blocktype.
|          string form ["vertex", "edge", "face",  "facet", "vertexedge",
|          or list of strings for combining multiple blocktypes
|          int is for backward compatibility with old style blocktypes
|
|      condense: bool = False
|          exclude dofs eliminated by static condensation
|
|  Elements(...)
|      Elements(*args, **kwargs)
|      Overloaded function.
|
|      1. Elements(self: ngsolve.comp.FESpace, VOL_or_BND: ngsolve.comp.V
|
|
|      Returns an iterable range of elements.
|
|      Parameters:
|
|      VOL_or_BND : ngsolve.comp.VorB
|        input VOL, BND, BBND,...
|
|
|
|      2. Elements(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.Region)
|
|  FinalizeUpdate(...)
|      FinalizeUpdate(self: ngsolve.comp.FESpace) -> None
|
|      finalize update
|
|  FreeDofs(...)
|      FreeDofs(self: ngsolve.comp.FESpace, coupling: bool = False) -> py
|
|
|
|      Return BitArray of free (non-Dirichlet) dofs\n
|      coupling=False ... all free dofs including local dofs\n
|      coupling=True .... only element-boundary free dofs
|
|      Parameters:
|
|      coupling : bool
|        input coupling
|
|  GetDefinedOnRegion(...)
|      GetDefinedOnRegion(self: ngsolve.comp.FESpace, vb: ngsolve.comp.VO
|
```

```
|      Return mesh-region where FESpace is defined on
|
|  GetDirichletRegion(...)
|      GetDirichletRegion(self: ngsolve.comp.FESpace, vb: ngsolve.comp.V
|
|      Return boundary Dirichlet mesh-region of FESpace
|
|  GetDofNrs(...)
|      GetDofNrs(*args, **kwargs)
|      Overloaded function.
|
|      1. GetDofNrs(self: ngsolve.comp.FESpace, ei: ngsolve.comp.Element
|
|
|
|      Parameters:
|
|      ei : ngsolve.comp.ElementId
|        input element id
|
|
|
|      2. GetDofNrs(self: ngsolve.comp.FESpace, ni: ngsolve.comp.NodeId)
|
|
|
|      Parameters:
|
|      ni : ngsolve.comp.NodeId
|        input node id
|
|  GetDofs(...)
|      GetDofs(self: ngsolve.comp.FESpace, region: ngsolve.comp.Region) -
|
|
|      Returns all degrees of freedom in given region.
|
|      Parameters:
|
|      region : ngsolve.comp.Region
|        input region
|
|  GetFE(...)
|      GetFE(self: ngsolve.comp.FESpace, ei: ngsolve.comp.ElementId) -> 
|
|
|      Get the finite element to corresponding element id.
|
|      Parameters:
|
|      ei : ngsolve.comp.ElementId
|         input element id
|
|  GetNDofLevel(...)
```

```
|      GetNDofLevel(self: ngsolve.comp.FESpace, level: typing.SupportsInt
|
|  GetOrder(...)
|      GetOrder(self: ngsolve.comp.FESpace, nodeid: ngsolve.comp.NodeId)
|
|      return order of node.
|      by now, only isotropic order is supported here
|
|  GetTrace(...)
|      GetTrace(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.FESpace, a
|
|  GetTraceTrans(...)
|      GetTraceTrans(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.FESpa
|
|  HideAllDofs(...)
|      HideAllDofs(self: ngsolve.comp.FESpace, component: object = <ngsol
|
|      set all visible coupling types to HIDDEN_DOFs (will be overwritten
|
|  InvM(...)
|      InvM(self: ngsolve.comp.FESpace, rho: ngsolve.fem.CoefficientFunct
|
|  Mass(...)
|      Mass(self: ngsolve.comp.FESpace, rho: ngsolve.fem.CoefficientFunct
|
|  ParallelDofs(...)
|      ParallelDofs(self: ngsolve.comp.FESpace) -> ngsolve.la.ParallelDof
|
|      Return dof-identification for MPI-distributed meshes
|
|  Prolongation(...)
|      Prolongation(self: ngsolve.comp.FESpace) -> ngmg::Prolongation
|
|      Return prolongation operator for use in multi-grid
|
|  Range(...)
|      Range(self: ngsolve.comp.FESpace, arg0: typing.SupportsInt | typin
|
|      deprecated, will be only available for ProductSpace
|
|  SetCouplingType(...)
|      SetCouplingType(*args, **kwargs)
|      Overloaded function.
|
|      1. SetCouplingType(self: ngsolve.comp.FESpace, dofnr: typing.Suppo
|
|
|               Set coupling type of a degree of freedom.
|
|      Parameters:
|
|      dofnr : int
|        input dof number
|
```

```
|      coupling_type : ngsolve.comp.COUPLING_TYPE
|        input coupling type
|
|
|
|      2. SetCouplingType(self: ngsolve.comp.FESpace, dofnrs: ngsolve.ngs
|
|
|               Set coupling type for interval of dofs.
|
|      Parameters:
|
|      dofnrs : Range
|        range of dofs
|
|      coupling_type : ngsolve.comp.COUPLING_TYPE
|        input coupling type
|
|  SetDefinedOn(...)
|      SetDefinedOn(self: ngsolve.comp.FESpace, region: ngsolve.comp.Regi
|
|
|      Set the regions on which the FESpace is defined.
|
|      Parameters:
|
|      region : ngsolve.comp.Region
|        input region
|
|  SetHarmonicProlongation(...)
|      SetHarmonicProlongation(self: ngsolve.comp.FESpace, bf: ngcomp::Bi
|
|      use harmonic prolongation w.r.t. bilinear-form
|
|  SetOrder(...)
|      SetOrder(*args, **kwargs)
|      Overloaded function.
|
|      1. SetOrder(self: ngsolve.comp.FESpace, element_type: ngsolve.fem.
|
|
|
|      Parameters:
|
|      element_type : ngsolve.fem.ET
|        input element type
|
|      order : object
|        input polynomial order
|
|
|      2. SetOrder(self: ngsolve.comp.FESpace, nodeid: ngsolve.comp.NodeI
|
|
```

```
|
|      Parameters:
|
|      nodeid : ngsolve.comp.NodeId
|        input node id
|
|      order : int
|        input polynomial order
|
|  SolveM(...)
|      SolveM(self: ngsolve.comp.FESpace, vec: ngsolve.la.BaseVector, rho
|
|
|               Solve with the mass-matrix. Available only for L2-like sp
|
|      Parameters:
|
|      vec : ngsolve.la.BaseVector
|        input right hand side vector
|
|      rho : ngsolve.fem.CoefficientFunction
|        input CF
|
|  TestFunction(...)
|      TestFunction(self: ngsolve.comp.FESpace) -> object
|
|      Return a proxy to be used as a testfunction for Symbolic Integrato
|
|  TnT(...)
|      TnT(self: ngsolve.comp.FESpace) -> tuple[object, object]
|
|      Return a tuple of trial and testfunction
|
|  TraceOperator(...)
|      TraceOperator(self: ngsolve.comp.FESpace, tracespace: ngsolve.comp
|
|  TrialFunction(...)
|      TrialFunction(self: ngsolve.comp.FESpace) -> object
|
|      Return a proxy to be used as a trialfunction in Symbolic Integrato
|
|  Update(...)
|      Update(self: ngsolve.comp.FESpace) -> None
|
|      update space after mesh-refinement
|
|  UpdateDofTables(...)
|      UpdateDofTables(self: ngsolve.comp.FESpace) -> None
|
|      update dof-tables after changing polynomial order distribution
|
|  __eq__(...)
|      __eq__(self: ngsolve.comp.FESpace, space: ngsolve.comp.FESpace) ->
|
```

```
 |  __mul__(...)
 |      __mul__(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.FESpace) -:
 |
 |  __pow__(...)
 |      __pow__(self: ngsolve.comp.FESpace, arg0: typing.SupportsInt | typ
 |
 |  __str__(...)
 |      __str__(self: ngsolve.comp.FESpace) -> str
 |
 |  __timing__(...)
 |      __timing__(self: ngsolve.comp.FESpace) -> object
 |
 |  ----------------------------------------------------------------------
 |  Static methods inherited from FESpace:
 |
 |  __special_treated_flags__(...) method of pybind11_builtins.pybind11_de
 |      __special_treated_flags__() -> dict
 |
 |  ----------------------------------------------------------------------
 |  Readonly properties inherited from FESpace:
 |
 |  autoupdate
 |
 |  components
 |      deprecated, will be only available for ProductSpace
 |
 |  couplingtype
 |
 |  dim
 |      multi-dim of FESpace
 |
 |  globalorder
 |      query global order of space
 |
 |  is_complex
 |
 |  loembedding
 |
 |  lospace
 |
 |  mesh
 |      mesh on which the FESpace is created
 |
 |  ndof
 |      number of degrees of freedom
 |
 |  ndofglobal
 |      global number of dofs on MPI-distributed mesh
 |
 |  type
 |      type of finite element space
 |
 |  ----------------------------------------------------------------------
 |  Data and other attributes inherited from FESpace:
```

```
 |
 |  __hash__ = None
 |
 |  ----------------------------------------------------------------------
 |  Readonly properties inherited from NGS_Object:
 |
 |  __memory__
 |
 |  flags
 |
 |  ----------------------------------------------------------------------
 |  Data descriptors inherited from NGS_Object:
 |
 |  name
 |
 |  ----------------------------------------------------------------------
 |  Static methods inherited from pybind11_builtins.pybind11_object:
 |
 |  __new__(*args, **kwargs) class method of pybind11_builtins.pybind11_ob
 |      Create and return a new object.  See help(type) for accurate signa
```